



Proyecto DEMETER

Plataforma Integral de Monitorización y Gestión
para Investigación Agrícola

IoT · Firmware ESP32 · Backend FastAPI · LIMS

Informe de Viabilidad y Cierre

Análisis de Sostenibilidad y Propuesta de Cierre Formal

Autor: Daniel Montero Fernández

Contacto: danmonfer@us.es

Asociación: ESIBot

Institución: Universidad de Sevilla

Versión: 1.0

Fecha: Agosto de 2026

Estado: Cierre formal propuesto

Documento complementario a la *Documentación Técnica del Sistema*.

Este documento es de uso interno del proyecto y de la colaboración institucional.

Índice

Resumen Ejecutivo	4
1 Introducción y Contexto	5
1.1 Proyecto Demeter — síntesis	5
1.2 Marco institucional	5
1.3 Propósito de este informe	6
2 Logros del Proyecto	7
2.1 Hitos técnicos alcanzados	7
2.1.1 Capa de Campo (Firmware ESP32)	7
2.1.2 Capa de Borde (Raspberry Pi)	7
2.1.3 Capa de Servidor (Google Cloud)	7
2.1.4 Capa de Visualización (React)	8
2.1.5 SDK Científico (Python)	8
2.2 Documentación y artefactos generados	8
2.3 Aprendizajes adquiridos	9
3 Estado Actual del Sistema	10
3.1 Componentes funcionales verificados	10
3.2 Componentes parciales o no implementados	11
3.3 Tests y validación realizados	11
3.3.1 Tests unitarios por capa	11
3.3.2 Tests de integración	12
3.3.3 Pruebas de campo	12
3.3.4 Métricas medidas	12
4 Análisis de Sostenibilidad	13
4.1 Análisis económico	13
4.1.1 Inversión realizada hasta la fecha	13
4.1.2 Coste de mantenimiento y operación	15
4.2 Análisis de recursos humanos	16
4.2.1 Composición actual de la asociación	16
4.2.2 Requisitos de equipo para mantenimiento sostenido	16
4.2.3 Single Point of Failure	17
4.3 Análisis técnico de mantenimiento	17
4.3.1 Complejidad del stack	17
4.3.2 Carga de mantenimiento estimada	17
4.3.3 Deuda técnica acumulada	18

5 Razones de No Sostenibilidad	19
5.1 Resumen estructural	19
5.2 ¿Por qué cerrar ahora y no antes?	20
6 Lecciones Aprendidas	21
6.1 Lecciones técnicas	21
6.2 Lecciones organizativas	22
6.3 Lecciones de gestión	22
7 Propuesta de Cierre	23
7.1 Opciones contempladas	23
7.2 Opción recomendada: combinación A + D	23
7.2.1 Concreción de la propuesta	24
7.3 Plan de entrega de artefactos	24
8 Recomendaciones para Futuros Proyectos	25
8.1 Dimensionar al equipo realmente disponible	25
8.2 Distribuir el conocimiento desde el inicio	25
8.3 Definir criterios de cierre desde el inicio	25
8.4 MVP iterativo > arquitectura aspiracional	25
8.5 Documentar para audiencias distintas	26
8.6 Reconocer los límites de la asociación estudiantil	26
A Detalle Financiero	27
A.1 Inventario de hardware adquirido	27
A.2 Histórico de gastos en infraestructura cloud	29
A.3 Desglose detallado de horas-persona	29
A.4 Fuentes de financiación recibidas	30
A.5 Plan de devolución / cesión de hardware	30
B Plan de Entrega de Artefactos	32
B.1 Artefactos digitales	32
B.2 Artefactos físicos	33
B.3 Artefactos de conocimiento e infraestructura	34
B.4 Cronograma detallado	35
B.5 Documento de cierre formal	36

Resumen Ejecutivo

El Proyecto Demeter, desarrollado por la Asociación Universitaria ESIBot en colaboración con la Universidad de Sevilla, ha alcanzado un estado funcional documentado que demuestra la viabilidad técnica de una plataforma IoT integral para investigación agrícola. A lo largo de **veinte meses** desde su inicio en diciembre de 2024 — de los cuales solo cuarenta y cuatro días corresponden a desarrollo activo, con 227 *commits* concentrados entre enero y marzo de 2026 — el sistema ha llegado a cubrir las cuatro capas proyectadas (firmware ESP32, *edge computing* en Raspberry Pi, backend en Google Cloud y frontend web) con documentación técnica completa de 84 páginas y validación por capa mediante 140 tests unitarios y de integración.

Sin embargo, el análisis de sostenibilidad del proyecto en su forma actual revela limitaciones estructurales que motivan su cierre formal: (1) la complejidad multistack del sistema requiere un equipo multidisciplinar sostenido en el tiempo que excede la capacidad actual de la asociación; (2) los recursos económicos necesarios para el mantenimiento operativo continuo superan las fuentes de financiación disponibles; (3) el desarrollo se ha concentrado en un único miembro, configurando un punto de fallo estructural incompatible con la rotación natural de una asociación estudiantil.

Este informe propone el cierre formal del proyecto en su forma actual con preservación completa del conocimiento generado: publicación del repositorio bajo licencia abierta, conservación de la documentación técnica como caso de estudio, y disponibilidad para reactivación parcial mediante TFG/TFM futuros sobre componentes específicos. Esta estrategia permite cumplir los objetivos didácticos del proyecto, preservar el valor académico generado, y mantener abierta la relación de colaboración con la Universidad para futuras iniciativas.

Capítulo 1

Introducción y Contexto

1.1 Proyecto Demeter — síntesis

Demeter nace en **diciembre de 2024** como iniciativa de la Asociación Universitaria ESIBot para abordar la necesidad de herramientas asequibles de monitorización ambiental e investigación agrícola de precisión. El sistema, definido por su arquitectura de cuatro capas, integra hardware embebido (ESP32), computación en el borde (Raspberry Pi), infraestructura cloud (Google Cloud Platform) y una interfaz web reactiva, articulados por un protocolo binario propietario optimizado para nodos de bajo consumo.

El propósito original no era construir una plataforma IoT genérica, sino resolver un problema concreto y bien acotado: en el **Grado en Ingeniería Agrícola** hay prácticas docentes cuyas plantas deben regarse y vigilarse a diario, y ese riego lo venían haciendo a mano los técnicos del Departamento de Agronomía. Demeter debía **automatizar ese riego** y liberar así horas de trabajo manual repetitivo, aportando de paso la trazabilidad de cada planta y cada experimento que un protocolo agronómico riguroso exige y que una libreta de campo no da.

Conviene retener el dato temporal, porque enmarca todo el análisis de este informe: entre esa fecha de nacimiento y la redacción del presente documento median **veinte meses**, de los cuales el desarrollo efectivo del sistema ocupa **cuarenta y cuatro días naturales** (227 *commits* entre enero y marzo de 2026). El resto del tiempo el proyecto estuvo detenido, no por falta de voluntad, sino por la ausencia de personas disponibles para empujarlo. Esa proporción entre tiempo transcurrido y tiempo trabajado es, en sí misma, la primera evidencia del problema de sostenibilidad que el capítulo 4 analiza en detalle.

La descripción técnica completa del sistema se recoge en la *Documentación Técnica del Sistema* (documento hermano de 84 páginas). El presente informe se centra en el análisis de viabilidad y la propuesta de cierre del proyecto.

1.2 Marco institucional

ESIBot es una asociación universitaria con sede en la **Escuela Técnica Superior de Ingeniería (ETSI)** de la Universidad de Sevilla. Reúne aproximadamente a **cien miembros** y trabaja en cuatro líneas: **robótica, electrónica, telemática y diseño 3D**. Su actividad se organiza en proyectos internos de duración variable, en los que los miembros participan de forma voluntaria y en paralelo a sus estudios.

El Proyecto Demeter se enmarcó como Proyecto Docente bajo el acuerdo de colaboración con el **Departamento de Agronomía de la Escuela Técnica Superior de Ingeniería Agronómica (ETSIA)**. El objetivo pactado era proveer un **sistema de riego automatizado** que sustituyera el trabajo manual de los técnicos encargados de las plantas asignadas a una práctica docente del Grado en Ingeniería Agrícola, añadiendo el registro y la trazabilidad de las variables ambientales de cada planta.

Merece la pena subrayar la naturaleza **interesescuela** de la colaboración: los desarrolladores pertenecían a la ETSI y el problema y sus beneficiarios estaban en la ETSIA. Esa distancia, que enriquecía el proyecto en lo formativo, tuvo también un coste: ninguna de las dos escuelas tenía el proyecto plenamente como propio, y la instalación física — que debía montarse en dependencias de la ETSIA con material adquirido por la ETSIA, pero por personas de la ETSI — nunca llegó a ejecutarse.

1.3 Propósito de este informe

El presente documento tiene tres objetivos:

1. Reportar el estado actual del proyecto y los logros alcanzados, con la honestidad técnica que la colaboración institucional exige.
2. Analizar la sostenibilidad del proyecto en su forma actual, identificando las limitaciones estructurales encontradas y aportando datos económicos, organizativos y técnicos concretos.
3. Proponer una estrategia de cierre que preserve el valor generado, cumpla los objetivos docentes pactados, y mantenga abiertas las opciones de continuidad bajo nuevos formatos.

El informe se ha estructurado de modo que sea útil tanto para una lectura completa por parte del responsable técnico como para una consulta rápida por parte de un evaluador académico: el resumen ejecutivo y la sección de propuesta de cierre (capítulo 7) contienen toda la información de decisión necesaria.

Capítulo 2

Logros del Proyecto

2.1 Hitos técnicos alcanzados

El sistema cubre las cuatro capas proyectadas y un SDK científico independiente. A continuación se enumeran los hitos verificados por capa.

2.1.1 Capa de Campo (Firmware ESP32)

- Implementación completa de los tres roles (Sensor, Actuador, Gateway) con código compartido y comportamiento configurable en compilación.
- Protocolo binario Demeter v2 funcional: **14** tipos de mensaje operativos, handshake TCP-like (SYN / SYN-ACK / ACK), gestión de reintentos.
- Operación en *Deep Sleep* para nodos sensor (~ 80 mA en activo, ~ 0.01 mA en reposo).
- Tabla de rutas dinámica con comando `ROUTE_ADD`.
- Pipeline de validación por capas: SYNC + LENGTH + CRC + DST_ID.

2.1.2 Capa de Borde (Raspberry Pi)

- Servicio Python `asyncio` con cuatro tareas concurrentes.
- Decodificación bidireccional Protocolo Demeter $\leftrightarrow$ JSON.
- Cliente WebSocket TLS persistente con reconexión automática.
- Containerización Docker con mapeo directo del UART físico.
- Caché local de topología (*Device Manager*) persistente.

2.1.3 Capa de Servidor (Google Cloud)

- Stack de seis contenedores en red Docker privada (`demeter-net`).
- Backend FastAPI con **31** endpoints REST (19 GET, 10 POST, 1 PATCH, 1 DELETE) y canal WebSocket.
- Persistencia PostgreSQL sobre **13** tablas, con migraciones gestionadas vía Alembic.
- Bus Redis Pub/Sub con dos bases de datos lógicas (caché + cola).
- Worker asíncrono para cálculos pesados (RQ).

- Secuenciador para automatización temporizada.
- Autenticación JWT con `bcrypt`.
- Sistema LIMS con plantas, experimentos y asignaciones N:M.

2.1.4 Capa de Visualización (React)

- *Single Page Application* con dashboard tiempo real.
- Control manual de actuadores.
- Planificador de secuencias multi-paso.
- Gestión LIMS: plantas, experimentos, asignaciones, consulta de telemetría por planta.

2.1.5 SDK Científico (Python)

- Cliente con caché Parquet automática (`~/.demeter_cache`).
- Pipeline ETL: `to_dataframe` $\rightarrow$ `clean` $\rightarrow$ `fill_gaps` $\rightarrow$ `resample`.
- Cálculos agronómicos vectorizados:
 - Termodinámica del aire húmedo: VPD, punto de rocío, bulbo húmedo (Stull, 2011), humedad absoluta, entalpía (ASHRAE).
 - Fenología: GDD, *chill hours*, ET_0 (Hargreaves–Samani), DAP.
 - Estadística: Z-score, índice de Wallin (1962), media móvil 24 h.
- Visualización con Matplotlib/Seaborn (4 funciones gráficas).
- Exportación CSV (UTF-8 BOM) y Excel multi-hoja.
- Suite completa en una sola llamada (`run_full_suite()`).

2.2 Documentación y artefactos generados

- Documentación técnica de arquitectura: **84 páginas** en LaTeX, más las 14.575 líneas de documentación en Markdown del árbol `docs/`.
- **18** diagramas distintos de flujo, secuencia y arquitectura (compilados en variante clara y oscura).
- **9** cuadros de *Decisión de Diseño* con justificación y alternativas descartadas.
- Repositorio Git con **227 commits** y **27.025** líneas de código propio, excluidas dependencias y ficheros generados.
- Suite de **140 tests** automatizados repartidos en 20 ficheros:
 - Firmware ESP32 (Unity/C++): 32 tests.
 - Servicio Raspberry Pi (pytest): 37 tests.
 - Backend FastAPI (pytest): 16 tests.
 - Frontend (Vitest): 3 tests.
 - SDK científico (pytest): 52 tests.
- Esquema de datos PostgreSQL versionado con Alembic.
- Modelos Pydantic compartidos entre todas las capas.

2.3 Aprendizajes adquiridos

Los miembros participantes han desarrollado competencias en:

- Diseño de protocolos de comunicación binarios.
- Programación embebida ESP32 + ESP-NOW.
- Programación asíncrona en Python (`asyncio`).
- Backend moderno (FastAPI, asincronía, Pydantic, SQLAlchemy asíncrona).
- Frontend reactivo (React, WebSockets en cliente).
- DevOps básico (Docker, despliegue cloud, gestión de secretos).
- Documentación técnica profesional (LaTeX, diagramas).
- Cálculos agronómicos vectorizados (NumPy/Pandas).
- Especificación y redacción de decisiones de arquitectura (ADR) y de exploraciones acotadas (*spikes*) como herramientas de trabajo, no como burocracia.
- Evaluación crítica de la sostenibilidad de un proyecto propio y capacidad de proponer su cierre ordenado, que es en sí misma una competencia de ingeniería.

Capítulo 3

Estado Actual del Sistema

3.1 Componentes funcionales verificados

A fecha de redacción del presente informe, los siguientes componentes del sistema operan de forma verificada en entorno de pruebas:

- **Telemetría extremo a extremo.** Un nodo sensor ESP32-S3 despierta de *deep sleep*, lee sus sondas DS18B20 y sus sensores capacitivos de humedad de suelo, emite un `SENSOR_CLUSTER_REPORT` por ESP-NOW al *gateway*, éste lo reenvía por UART a la Raspberry Pi, que lo decodifica y lo publica por WebSocket al backend, donde queda persistido en PostgreSQL y visible en el dashboard.
- **Comando manual extremo a extremo.** La cadena dashboard → POST `/api/command` → Redis Pub/Sub → *dispatcher* WebSocket → Raspberry Pi → UART → *gateway* → ESP-NOW → nodo actuador opera de forma completa para el comando `SET_GPIO`.
- **Handshake y enrutado.** Establecimiento de sesión SYN / SYN-ACK / ACK entre nodo y *gateway*, y alta de rutas en caliente mediante `ROUTE_ADD`.
- **Secuencias programadas.** El servicio secuenciador ejecuta secuencias multi-paso definidas desde el planificador del frontend.
- **Trazabilidad LIMS.** Alta de plantas y experimentos, asignación N:M entre ambos y consulta de telemetría filtrada por planta.
- **Autenticación.** Registro e inicio de sesión con JWT y `bcrypt`, con protección efectiva de las rutas privadas.
- **SDK científico.** Extracción de series a `DataFrame` con caché Parquet, limpieza y remuestreo, cálculos agronómicos vectorizados y exportación a CSV y Excel.
- **Despliegue reproducible.** Los cinco entornos de *Docker Compose* (desarrollo, *override*, *staging*, producción y Raspberry Pi) levantan desde un árbol limpio, y la integración continua construye y publica las imágenes de backend y frontend en GitHub Container Registry.

Escenario de prueba más representativo. La validación integrada se realizó sobre banco de laboratorio con **tres nodos ESP32-S3** (uno *gateway* conectado por UART a la Raspberry Pi y dos nodos de campo) y hasta dos plantas instrumentadas por nodo, con un ciclo de *deep sleep* de 60 segundos. El procedimiento completo, incluido el conexionado, está documentado en `docs/deep_sleep_test.md` y es reproducible. Es importante precisar el alcance de esta validación: se trata de **sesiones de prueba en laboratorio, no de una campaña de operación continua**. El sistema nunca llegó a desplegarse en un invernadero real ni a funcionar sin supervisión durante días.

3.2 Componentes parciales o no implementados

Por honestidad técnica y para facilitar la replicabilidad o la continuación parcial del proyecto, se documentan los aspectos que se encuentran en estado parcial o pendiente:

- **Cifrado en la capa de campo.** AES-128 sobre el *payload* ESP-NOW estaba planificado para la versión 3.0 y no está implementado. La trama es legible por cualquier ESP32 cercano sintonizado al canal.
- **Discovery dinámico incompleto.** Existe un *Device Manager* con caché persistente de topología y un *router* de descubrimiento en el backend, pero el alta de un nodo nuevo no es autónoma: la dirección MAC del *gateway* está fijada en tiempo de compilación en el firmware de los nodos y el `NODE_ID` se asigna también por *build flag*. Añadir un nodo exige recompilar y reflashear.
- **Sin observabilidad.** No hay agregación de *logs*, métricas, alarmas ni notificaciones. El diagnóstico de una incidencia obliga a leer la salida de cada contenedor por separado.
- **Dashboard no adaptado a móvil.** La interfaz está diseñada para pantalla de escritorio.
- **Riego físico sin validar.** La instalación hidráulica adquirida (capítulo 4) no llegó a montarse. El *spike* de validación de bombeo (`docs/spikes/SPIKE-001`) quedó escrito y sin ejecutar, y además está redactado para una bomba de 5 V mientras que el material comprado es de 12 V, por lo que su plan de pruebas tampoco sería aplicable sin reescribirlo.

Conviene señalar que **el código no contiene marcas TODO ni FIXME significativas**: no hay funciones a medio escribir ni parches provisionales pendientes de resolver. Lo que falta no está dentro de lo construido, sino que son capas que nunca se empezaron. Es deuda de alcance, no de implementación.

Se considera que ninguna de estas limitaciones impide la utilidad docente ni el aprovechamiento del sistema como caso de estudio: la arquitectura está documentada y los puntos parciales están claramente identificados.

3.3 Tests y validación realizados

La validación del sistema se ha articulado en múltiples niveles.

3.3.1 Tests unitarios por capa

Cada capa cuenta con suite de tests unitarios independiente:

Cuadro 3.1: Tests unitarios por capa.

Capa	Framework	Número de tests
Firmware ESP32	Unity (C++)	32
Servicio Raspberry Pi	pytest	37
Backend FastAPI	pytest	16
Frontend	Vitest	3
SDK científico	pytest	52
TOTAL		140

Sobre la cobertura. La tabla no incluye porcentaje de cobertura porque **el proyecto nunca instrumentó su medida**: no hay configuración de `coverage` ni umbral mínimo en integración continua, y ningún informe histórico que citar. Se ha preferido declararlo abiertamente antes que producir

ahora una cifra retrospectiva que no represente el estado del proyecto durante su desarrollo. El reparto de los tests es, en sí mismo, informativo: las capas con lógica algorítmica pura y fácilmente aislable (SDK científico, protocolo del firmware, caché de telemetría) concentran la mayor parte de las pruebas, mientras que las capas dominadas por integración con servicios externos (backend, frontend) quedan claramente por debajo.

3.3.2 Tests de integración

Además de los tests unitarios se mantienen:

- Un entorno de integración del firmware (`pio test -e integration`) que ejercita los flujos completos del `SystemManager` y del `ProtocolEngine` sobre *mocks* de Arduino, sin hardware.
- Tests de backend sobre cliente HTTP de prueba que recorren autenticación, emisión de comandos, LIMS y análisis contra una base de datos real levantada en contenedor.
- Validación manual extremo a extremo sobre el entorno de *staging*, con la Raspberry Pi conectada al backend remoto y los tres nodos operativos.

3.3.3 Pruebas de campo

No se realizaron. El sistema se validó únicamente sobre banco de laboratorio, en el escenario descrito al principio de este capítulo. No hubo instalación en invernadero, ni operación desatendida prolongada, ni exposición de los nodos a condiciones ambientales reales.

3.3.4 Métricas medidas

El sistema no llegó a caracterizarse cuantitativamente. No se midieron la latencia extremo a extremo, la tasa de pérdida de tramas ESP-NOW, la vida útil real de un nodo alimentado por batería, el *throughput* sostenido de telemetría ni el tiempo medio de reconexión del WebSocket. Los únicos valores de consumo que aparecen en este informe (~ 80 mA en activo, $\sim 0,01$ mA en reposo) proceden de la hoja de características del ESP32-S3, no de una medida propia.

Se ha optado por declarar esta ausencia en lugar de rellenar una tabla con estimaciones. La falta de caracterización no es un descuido aislado: es exactamente el tipo de trabajo que un proyecto sostenido por una sola persona en horario nocturno termina posponiendo, y como tal refuerza el análisis del capítulo 4 en lugar de contradecirlo. Cualquier continuación del proyecto debería empezar por aquí: sin línea base medida no hay forma de saber si un cambio mejora o empeora el sistema.

Capítulo 4

Análisis de Sostenibilidad

Este capítulo analiza la viabilidad de mantener el Proyecto Demeter operativo en su forma actual a lo largo del tiempo, examinando las tres dimensiones críticas: económica, de recursos humanos y técnica.

4.1 Análisis económico

4.1.1 Inversión realizada hasta la fecha

El proyecto ha consumido recursos económicos en hardware embebido, infraestructura cloud y horas de desarrollo personal. Las siguientes tablas detallan estos costes.

Cuadro 4.1: Inversión en electrónica de control y sensores.

Componente	Uds.	€/ud.	Total
ESP32-S3-DevKitC-1	3	~15€	45€
Raspberry Pi 4B (4 GB)	1	~75€	75€
MicroSD 32 GB clase 10	1	~9€	9€
Sensor DS18B20 (sonda impermeable)	10	~4€	40€
Sensor capacitivo de humedad de suelo	10	~3€	30€
Sensor DHT22	10	~5€	50€
Módulo relé 4 canales	3	~6€	18€
Convertidores TTL/USB	2	~5€	10€
Fuentes de alimentación 5 V	3	~5€	15€
Baterías LiPo 3000 mAh	2	~12€	24€
Carcasas/protecciones IP65	2	~10€	20€
Cables, jumpers, <i>breadboard</i>	—	—	30€
SUBTOTAL ELECTRÓNICA	—	—	366€

La electrónica se adquirió de forma incremental en distribución *online* a lo largo del desarrollo, en pedidos pequeños y sin un expediente de compra unificado; los importes de la tabla 4.1 son por tanto precios de referencia de mercado por unidad, no importes facturados uno a uno. La instalación hidráulica, por el contrario, sí procede de un único expediente con trazabilidad completa.

Cuadro 4.2: Inversión en instalación hidráulica y de potencia (presupuesto nº 26317, MR MICRO Sevilla).

Componente	Uds.	€/ud.	Total
Bomba eléctrica autocebada 12 V (Coolty)	4	29,99 €	119,96 €
Electroválvula DC 12 V 1 "	6	36,74 €	220,44 €
Válvula reguladora de presión 1 "	4	31,91 €	127,64 €
Válvula antirretorno 1 "	4	18,11 €	72,44 €
Barril 60 L boca ancha HDPE sin BPA	4	28,45 €	113,80 €
Kit de grifos para tanque	2	16,80 €	33,60 €
Fuente de alimentación carril DIN 12 V 120 W	1	29,93 €	29,93 €
Cable plano gemelo 2 núcleos 10 m	1	25,99 €	25,99 €
Base imponible	—	—	743,80 €
IVA (21 %)	—	—	156,20 €
SUBTOTAL HIDRÁULICA	—	—	900,00 €

Cuadro 4.3: Inversión total en hardware.

Bloque	Importe
Electrónica de control y sensores (tabla 4.1)	366 €
Instalación hidráulica y de potencia (tabla 4.2)	900 €
TOTAL HARDWARE	1.266 €

Cuadro 4.4: Infraestructura cloud (Google Cloud Platform).

Servicio	€/mes	Acumulado
VM e2-small (always-on)	~14 €	~112 €
Disco persistente 30 GB	~1,5 €	~12 €
IP estática reservada	~3 €	~24 €
Tráfico de salida (estimado)	~0 €	0 €
Dominio (.com, prorrateado)	~1 €	~8 €
Certificado SSL (Let's Encrypt)	0 €	0 €
Otros servicios y variación de uso	~5,5 €	~44 €
TOTAL CLOUD FACTURADO	25 €	200 €

El importe facturado real por Google Cloud Platform fue de **25 €/mes** a lo largo de los **ocho meses** de operación continua del entorno (enero-agosto de 2026), con un total definitivo de **200 €**. La cuenta ya ha sido cerrada, de modo que esa cifra no crecerá. Los **300 €/año** que se manejan más adelante en este capítulo no son, por tanto, un gasto en curso, sino el coste que habría que *volver a asumir* para mantener el sistema operativo. El desglose por servicio de la tabla 4.4 es orientativo y se ofrece para explicar la composición del gasto; el detalle mes a mes se recoge en el Anexo A.

Cuadro 4.5: Estimación del coste en horas-persona del desarrollo.

Fase	Horas estimadas
Investigación previa y diseño	15
Firmware ESP32	28
Servicio Raspberry Pi	10
Backend FastAPI	28
Frontend	15
SDK científico Python	10
Documentación técnica	22
Testing y depuración	12
Despliegue e integración	15
TOTAL HORAS DE DESARROLLO	~155 h

Procedencia de la cifra. El proyecto no dispuso de un sistema formal de registro de horas, por lo que la estimación se construye en dos pasos. Primero se calcula un **suelo objetivo** a partir de los sellos de tiempo de los 227 *commits* del repositorio, agrupados en sesiones de trabajo (una sesión termina tras dos horas sin *commits*, y se añaden 45 minutos por sesión para el trabajo previo al primer *commit*): 38 sesiones y ~87 horas. Ese suelo solo captura el tiempo próximo a un *commit* y deja fuera el montaje y cableado del hardware, la lectura de hojas de características, la depuración sin *commit* y buena parte de la redacción de las 84 páginas del manual técnico. Corrigiendo por esas partidas, la horquilla honesta se sitúa entre 120 y 180 horas, y la tabla 4.5 adopta el punto medio de 155 horas.

A modo orientativo, valorando estas horas a precio de mercado de desarrollador junior (20–30 €/h), el coste equivalente de externalización de un proyecto de esta envergadura sería del orden de 3.875 €.

4.1.2 Coste de mantenimiento y operación

Para sostener el sistema operativo de forma continua, los costes recurrentes mínimos son los siguientes:

Cuadro 4.6: Costes operativos recurrentes mínimos anuales.

Concepto	€/año
Infraestructura cloud GCP	~300€
Dominio y certificados	~12€
Reemplazo hardware (5 % anual)	~63€
Baterías nodos sensores	~24€
Margen de imprevistos (15 %)	~60€
TOTAL ANUAL ESTIMADO	~459€

Frente a este coste, la financiación disponible de manera recurrente es la siguiente:

Cuadro 4.7: Financiación recurrente disponible.

Fuente	€/año
Asignación universitaria (Proyecto Docente)	0€
Cuotas de la asociación	0€
Patrocinios externos	0€
TOTAL DISPONIBLE	0€

La única financiación institucional recibida por el proyecto fue la asignación del Proyecto Docente de la ETSIA que cubrió el expediente de 900 € de la tabla 4.2. Se trata de una **dotación puntual de inversión, no de una partida recurrente**: no financia gasto corriente ni está comprometida para ejercicios siguientes. La asociación no destina cuota alguna al mantenimiento del proyecto y no se han obtenido patrocinios externos. La financiación recurrente comprometida es, por tanto, nula.

Conclusión

El *gap* entre el coste operativo mínimo ($\sim 459 \text{ €/año}$) y la financiación recurrente disponible (0 €/año) es de 459 €/año , es decir, la totalidad del coste. Sin nuevas fuentes de financiación recurrente comprometidas, la operación continua del sistema desplegado no es viable.

4.2 Análisis de recursos humanos

4.2.1 Composición actual de la asociación

ESIBot, como toda asociación universitaria, opera con un modelo de **rotación natural**: los miembros entran al inicio de sus estudios y salen al graduarse, con permanencia media estimada de 2–4 años.

- Composición de la asociación: ~ 100 miembros, repartidos entre robótica, electrónica, telemática y diseño 3D.
- Miembros actualmente vinculados al Proyecto Demeter: **5**.
- Miembros con conocimiento profundo de Demeter: **1**.
- Miembros con capacidad de mantenimiento autónomo del sistema completo: **1**.

4.2.2 Requisitos de equipo para mantenimiento sostenido

El proyecto requiere conocimientos en al menos las siguientes áreas técnicas:

- Embedded C/C++ (ESP32, ESP-NOW, sensores, FreeRTOS).
- Python avanzado (`asyncio`, FastAPI, Pydantic).
- DevOps: Docker, despliegue cloud, monitorización.
- Frontend: React, WebSockets, gestión de estado.
- Bases de datos: PostgreSQL, Redis, migraciones.
- *Networking*: TLS, WebSocket, protocolos binarios.
- Cálculo agronómico (para mantener el SDK).

Estimación realista del equipo mínimo viable para mantener el sistema operativo sin desarrollo nuevo:

- 1 responsable de firmware (15–20 % de dedicación).
- 1 responsable de backend/DevOps (20–30 %).
- 1 responsable de frontend (10–15 %).
- 1 coordinador con visión transversal (10–15 %).

Esta carga representa aproximadamente el **60–80 % de un FTE** distribuido entre cuatro personas con dedicación parcial. Esta intensidad es **incompatible** con la disponibilidad real de los miembros actuales de la asociación, que combinan su actividad con estudios de grado/máster.

El contraste con las cifras de la sección anterior es directo: el mantenimiento exige **cuatro perfiles** distintos, y la asociación dispone hoy de **uno**. Conviene precisar dónde está exactamente el problema, porque no es donde parece. ESIBot tiene cien miembros: no falta gente, ni falta talento. Lo que falta es **conocimiento transferido**. De las cinco personas vinculadas al proyecto, una sola puede mantenerlo, y esa concentración no se corrige contratando ni reclutando, sino con meses de acompañamiento que nadie está en disposición de dar ni de recibir. Un sistema que solo una persona entre cien puede sostener no es un sistema de la asociación: es un sistema de esa persona alojado en la asociación.

4.2.3 Single Point of Failure

Hasta la fecha, el desarrollo se ha concentrado mayoritariamente en un único miembro (el autor de este informe). Esta concentración:

- **Facilitó** la coherencia arquitectónica del sistema y una velocidad de desarrollo elevada.
- Pero **configuró** un riesgo estructural significativo: al desvincularse este miembro, ningún otro tiene la capacidad actual para mantener el conjunto del sistema.

Este patrón (*bus factor* = 1) es común en proyectos estudiantiles ambiciosos y constituye una de las principales causas de fragilidad identificadas en la literatura sobre asociaciones universitarias.

4.3 Análisis técnico de mantenimiento

4.3.1 Complejidad del stack

Cuadro 4.8: Complejidad del stack tecnológico por capa.

Capa	Tecnologías principales	Complejidad
Firmware	C++, ESP-IDF, ESP-NOW, FreeRTOS	Alta
Edge (RPI)	Python, asyncio, Docker, UART async	Media
Backend	FastAPI, SQLAlchemy asíncrona, Redis, PostgreSQL	Alta
Frontend	React, WebSocket, gestión de estado	Media
DevOps	GCP, Docker, TLS, gestión de secretos	Media
Ciencia de datos	NumPy, Pandas, agronomía	Media

4.3.2 Carga de mantenimiento estimada

Operación en régimen normal, sin desarrollo nuevo:

- Monitorización del sistema: ~2–3 h/semana.
- Resolución de incidencias: ~3–5 h/semana.
- Actualizaciones de seguridad y dependencias: ~5–10 h/mes.
- Reemplazo y calibración de hardware: ~10–20 h/trimestre.

Total estimado: ~40–60 horas/mes $\approx$ **0,3 FTE**. Esta cifra coincide con la estimación de equipo mínimo de la sección 4.2.

4.3.3 Deuda técnica acumulada

A continuación se identifican aspectos que requerirían refactor o trabajo adicional para garantizar la sostenibilidad a medio plazo:

- **Cifrado ESP-NOW pendiente.** El *payload* viaja en claro; el cifrado AES-128 estaba planificado para la versión 3.0 y no llega a implementarse. Cualquier ESP32 sintonizado al canal puede leer las tramas.
- **Sin observabilidad centralizada.** No hay agregación de *logs* ni métricas: el diagnóstico de una incidencia obliga a entrar por SSH en la máquina y leer la salida de cada contenedor por separado.
- **CI/CD parcial.** El *pipeline* construye y publica imágenes de *backend* y *frontend*, pero no compila el firmware ni ejecuta análisis de secretos, y el despliegue sobre la máquina de producción sigue siendo manual.
- **Dependencias versionadas en el repositorio.** El árbol contiene `node_modules` y documentación HTML generada por Doxygen bajo control de versiones, además de ficheros de *log* subidos por error. No compromete el funcionamiento, pero infla el repositorio y distorsiona cualquier métrica automática sobre él.
- **Cobertura de tests no instrumentada.** Existen 140 tests (véase la tabla 3.1), pero no hay configuración de medida de cobertura ni umbral mínimo en integración continua.
- **Validación hidráulica pendiente.** El spike de validación de bombeo (`docs/spikes/SPIKE-001`) quedó escrito pero sin ejecutar, y además está redactado para una bomba de 5 V, mientras que la instalación efectivamente adquirida es de 12 V. La cadena de riego no llega a validarse extremo a extremo sobre el hardware real.

Merece una mención aparte que **el código no contiene marcas TODO ni FIXME significativas**: la deuda técnica de Demeter no está en funciones a medio escribir, sino en las capas que nunca se llegaron a construir (observabilidad, seguridad de campo, automatización del despliegue). Es una deuda de alcance, no de implementación.

Capítulo 5

Razones de No Sostenibilidad

5.1 Resumen estructural

A partir del análisis del capítulo 4, se identifican cuatro razones estructurales que motivan la no sostenibilidad del proyecto en su forma actual. Estas razones son **estructurales**, **no personales**, y comunes en proyectos estudiantiles que escalan en complejidad técnica.

R1. Escala humana

La complejidad multistack del sistema requiere un equipo multidisciplinar de aproximadamente cuatro personas con dedicación parcial (60–80 % FTE total distribuido). La rotación natural de miembros en una asociación estudiantil — con permanencia media de 2–4 años — no permite mantener ese nivel de capacidad técnica acumulada de forma sostenida sin un programa explícito de transferencia de conocimiento que la asociación no tiene actualmente capacidad de implementar.

R2. Escala económica

El coste operativo recurrente mínimo ($\sim 459 \text{€}/\text{año}$) excede por completo la financiación recurrente disponible ($0 \text{€}/\text{año}$). No se trata de un desfase parcial: **no existe ninguna partida comprometida** para el gasto corriente del proyecto. La única financiación institucional recibida — los 900 € del Proyecto Docente de la ETSIA — fue una dotación puntual de inversión en material, no una asignación recurrente. El *gap* requiere, por tanto, crear desde cero una fuente de ingresos sostenida que la asociación no puede asegurar bajo el formato actual.

Un dato ilustra el problema mejor que cualquier proyección: el último *commit* del repositorio es de marzo de 2026, pero la factura de Google Cloud siguió llegando cada mes durante cinco meses más, hasta que la cuenta se cerró de forma expresa. **El coste de un sistema desplegado no se detiene cuando se detiene el desarrollo**; solo se detiene cuando alguien lo apaga. Esa asimetría es precisamente lo que convierte una infraestructura sin equipo en un pasivo, y explica por qué el cierre ordenado que propone este informe no es una formalidad administrativa sino una medida con efecto económico inmediato.

R3. Escala de complejidad

El proyecto integra tecnologías de seis dominios distintos (*embedded*, *networking*, backend, frontend, DevOps, agronomía). Cada dominio aisladamente es manejable, pero la integración y mantenimiento simultáneo demandan especialización multidisciplinar difícil de mantener en el tiempo en un contexto estudiantil donde los miembros priorizan, lógicamente, sus estudios.

R4. Single Point of Failure

El desarrollo se ha concentrado en un único miembro, creando una dependencia estructural incompatible con la rotación natural de la asociación. La distribución real del conocimiento del sistema implica que, sin este miembro, el resto de la asociación no tiene capacidad operativa sobre el conjunto del sistema.

5.2 ¿Por qué cerrar ahora y no antes?

Estas limitaciones eran parcialmente conocidas desde el inicio del proyecto. La decisión de cierre formal **ahora**, frente a una continuación indefinida, responde a las siguientes consideraciones:

- **El sistema ha alcanzado un hito relevante:** la funcionalidad completa está documentada y validada por capa. El cierre se realiza **con valor demostrable**, no en estado intermedio.
- **La identificación clara de las limitaciones permite cierre razonado:** no es abandono por agotamiento, sino conclusión fundamentada en datos técnicos y económicos específicos.
- **Mantener el proyecto activo bajo condiciones no sostenibles generaría coste creciente sin valor proporcional**, y eventualmente daño reputacional al no poder cumplir expectativas operativas.
- **El cierre maduro preserva la relación institucional** y deja abiertas opciones de continuidad bajo nuevos formatos (caso de estudio docente, base para TFG/TFM, transferencia eventual a un grupo de investigación).

Consideración Importante

Reconocer cuándo un proyecto debe cerrar es una competencia profesional reconocida en la literatura de gestión de proyectos (Edmondson, 2023). En el contexto de proyectos docentes, esta capacidad de evaluación crítica es uno de los aprendizajes más valiosos que la experiencia ha aportado a los miembros participantes.

Capítulo 6

Lecciones Aprendidas

Este capítulo recoge las lecciones aprendidas durante el ciclo de vida del Proyecto Demeter, organizadas en tres dimensiones: técnicas, organizativas y de gestión. Estas lecciones constituyen un activo adicional del proyecto, aplicables tanto a futuros proyectos de la asociación como a otras iniciativas docentes similares.

6.1 Lecciones técnicas

- **Modelos compartidos como infraestructura central:** el uso de modelos Pydantic compartidos entre todas las capas para serialización/deserialización JSON y binaria resultó especialmente acertado. La misma definición de mensaje sirve para validación en backend y para empaquetado en firmware sin duplicación de lógica.
- **Filosofía de fallo silencioso en redes RF:** descartar silenciosamente las tramas corruptas evita tormentas de retroalimentación en presencia de interferencias. La lógica de aplicación aporta los ACK necesarios.
- **Cuadros de *Decisión de Diseño* en documentación:** documentar explícitamente las justificaciones de decisiones técnicas resulta crítico para la mantenibilidad y la incorporación de nuevos miembros. Las cajas con razonamiento implícito han demostrado ser una de las mejores aportaciones documentales.
- **Containerización Docker en Raspberry Pi:** si bien introduce overhead inicial, la consistencia de despliegue y la facilidad de actualizaciones justificó ampliamente la decisión.
- **Lección costosa: ausencia de cifrado en capa de campo:** planificar AES-128 como mejora futura en lugar de implementarlo desde el principio resultó ser deuda técnica significativa al considerar despliegues en entornos de investigación con datos sensibles.
- **Patrón que se repetiría: el modelo compartido entre capas.** Definir el protocolo una sola vez con modelos Pydantic en `Software/Common` y consumirlos desde el backend y desde la Raspberry Pi eliminó de raíz toda una clase de errores de desincronización entre extremos. Lo mismo aplica al patrón *Strategy* sobre la interfaz `IComms` del firmware, que permitió probar la lógica de protocolo en el PC sin hardware.
- **Decisión que se cambiaría: medir antes de optimizar y antes de cerrar.** El sistema se dio por bueno sin una línea base de latencia, pérdida de tramas ni autonomía real de los nodos. Sin esas medidas no hay forma de demostrar que funciona ni de detectar una regresión, y añadirlas al final ya no es posible. La instrumentación mínima — cobertura de tests y cuatro métricas de campo — debería haber existido desde la primera versión.

- **Decisión que se cambiaría: validar el hardware crítico antes de comprarlo.** El *spike* de validación de bombeo se escribió pero nunca se ejecutó, y para cuando se adquirió la instalación hidráulica (900 €) el plan de pruebas ya se había quedado obsoleto: estaba redactado para bombas de 5 V y el material comprado era de 12 V. El resultado es material sin desembalar. Un *spike* que no se ejecuta no reduce ningún riesgo; solo documenta que se era consciente de él.
- **Decisión que se cambiaría: higiene del repositorio desde el primer día.** Versionar `node_modules`, la documentación generada por Doxygen y ficheros de *log* parece inocuo al principio, pero infla el repositorio en un orden de magnitud y destruye la posibilidad de medir nada automáticamente sobre él. Un `.gitignore` correcto al inicio cuesta cinco minutos; corregirlo después implica reescribir el histórico.

6.2 Lecciones organizativas

- **Distribución del conocimiento desde el inicio:** la concentración del desarrollo en un único miembro fue la fuente principal de fragilidad del proyecto. Prácticas como pair programming, code reviews sistemáticas y rotación de responsabilidades habrían mitigado este *single point of failure* estructural.
- **Documentación viva durante el desarrollo, no al final:** la documentación técnica generada al final del proyecto es valiosa, pero documentar progresivamente durante el desarrollo habría facilitado la incorporación de nuevos miembros y la resolución más rápida de incidencias.
- **Onboarding sistematizado para nuevos miembros:** la asociación no contaba con un proceso definido de incorporación al proyecto. Una guía de *onboarding* habría reducido la curva de aprendizaje de los miembros que se incorporaron en fases intermedias.
- **Distinción entre documentación para evaluadores y documentación para mantenedores:** son audiencias y propósitos distintos. La documentación para evaluadores enfatiza arquitectura y decisiones; la documentación para mantenedores enfatiza cómo hacer cosas concretas.

6.3 Lecciones de gestión

- **Dimensionar las ambiciones al equipo realmente disponible:** el proyecto fue diseñado con una arquitectura de cuatro capas que requiere un equipo multidisciplinar consolidado. En retrospectiva, un MVP más acotado, con menos capas pero iterables, habría sido más compatible con la composición real de la asociación.
- **MVP iterativo > arquitectura aspiracional:** empezar con una versión mínima funcional end-to-end y añadir complejidad únicamente cuando se justifica con uso real es preferible a diseñar la arquitectura completa por anticipado.
- **Definir criterios de cierre desde el inicio:** cada proyecto debería tener criterios explícitos de éxito (cuándo se considera completado), pivote (cuándo cambiar de dirección) y cierre (cuándo concluir formalmente). El Proyecto Demeter no los tuvo formalmente; la decisión de cierre se toma ahora con criterios contruidos a posteriori.
- **Reconocer cuándo cerrar es competencia profesional:** saber decir “esto no podemos sostenerlo bajo las condiciones actuales” es parte del aprendizaje en gestión de proyectos. El presente informe es, en sí mismo, ejercicio de esa competencia.

Capítulo 7

Propuesta de Cierre

A partir de los análisis previos, este capítulo presenta cuatro opciones de cierre contempladas, la opción recomendada por el equipo y un plan de entrega concreto de los artefactos del proyecto.

7.1 Opciones contempladas

Opción A — Archivo limpio

Cierre completo y formal del proyecto, con publicación del repositorio en GitHub bajo licencia abierta y entrega de la documentación como material público. Cero coste de mantenimiento posterior.

Ventajas: simple, definitivo, sin compromisos pendientes. **Limitaciones:** no preserva opciones explícitas de continuidad.

Opción B — Cierre con opción de reactivación

Como Opción A, más un anexo que detalla qué condiciones permitirían una reactivación parcial del proyecto (financiación mínima requerida, equipo mínimo necesario, alcance reducido viable).

Ventajas: mantiene puerta abierta sin compromiso operativo. **Limitaciones:** si las condiciones nunca se dan, equivale a A.

Opción C — Transferencia a otro grupo

Cesión formal del proyecto a un grupo de investigación o a otro equipo capaz de mantenerlo. Requiere acuerdo explícito de transferencia, atribución y, eventualmente, soporte de los autores durante el traspaso.

Ventajas: el sistema sigue vivo bajo otra estructura. **Limitaciones:** requiere existencia de receptor interesado y mecanismos formales de transferencia.

Opción D — Reducción a caso docente

Conversión del proyecto en material didáctico estable: el sistema se documenta como caso de estudio reutilizable para asignaturas de IoT, sistemas embebidos, agronomía digital y gestión de proyectos. No requiere mantenimiento operacional ni infraestructura activa.

Ventajas: encaja perfectamente con el marco de Proyecto Docente original; cumple objetivos didácticos; cero coste recurrente. **Limitaciones:** el sistema deja de estar desplegado operativamente.

7.2 Opción recomendada: combinación A + D

El equipo propone articular el cierre como **combinación de archivo limpio (Opción A) y reducción a caso docente (Opción D)**, por las siguientes razones:

- Encaja con el marco original de Proyecto Docente bajo el que se obtuvo la financiación inicial.
- Cumple los objetivos didácticos planteados (desarrollo de competencias multidisciplinares, generación de material aprovechable docente).
- No requiere mantenimiento operacional ni financiación recurrente.
- Preserva el valor académico generado.
- Mantiene abierta la posibilidad de retomar componentes específicos como TFG/TFM futuros sobre piezas concretas (firmware, SDK, clúster cloud, etc.).

7.2.1 Concreción de la propuesta

1. **Repositorio público:** publicación completa del código en GitHub bajo licencia **MIT** para el código y **CC-BY-SA 4.0** para la documentación, con atribución a la Asociación ESIBot y a la Universidad de Sevilla.
2. **Documentación como material reutilizable:** la *Documentación Técnica del Sistema* (84 páginas) y el presente informe quedan disponibles, con licencia abierta, para cualquier uso docente que la Escuela o el Departamento consideren oportuno. No se propone aquí ninguna asignatura concreta: la decisión sobre dónde y cómo aprovechar el material corresponde a quien conozca los programas vigentes.
3. **Disponibilidad puntual de los autores:** el equipo se compromete a sesiones puntuales de presentación del sistema a futuros estudiantes o equipos interesados, durante un periodo de **doce meses** tras el cierre formal.
4. **Catálogo de TFG/TFM derivables:** se propone elaborar un documento breve (1–2 páginas) listando posibles trabajos derivables sobre componentes específicos del sistema, para uso de coordinadores docentes interesados.

7.3 Plan de entrega de artefactos

El cronograma de entrega propuesto se articula en cuatro semanas tras la aceptación formal del cierre, con los detalles técnicos recogidos en el Anexo [B](#).

Cuadro 7.1: Cronograma resumido de cierre.

Semana	Hito
Semana 1	Publicación del repositorio bajo licencia abierta.
Semana 2	Entrega formal de documentación (PDF + fuentes LaTeX).
Semana 3	Devolución / cesión del hardware adquirido.
Semana 4	Cierre de la infraestructura cloud (VM, dominio, IP).

El equipo solicita, como reconocimiento simbólico del cierre, un **acta de cierre formal** firmada por la Universidad y la Asociación, que recoja la conclusión del Proyecto Docente y los artefactos entregados. Este documento constituye reconocimiento del trabajo realizado y queda como referencia institucional de la colaboración.

Capítulo 8

Recomendaciones para Futuros Proyectos

A modo de cierre, este capítulo recoge seis recomendaciones derivadas de la experiencia del Proyecto Demeter, aplicables tanto a futuras iniciativas de la Asociación ESIBot como a otros proyectos docentes con características similares. Constituyen el último activo de valor del proyecto: la capitalización de la experiencia para terceros.

8.1 Dimensionar al equipo realmente disponible

No diseñar arquitecturas que requieran más equipo del que se puede sostener realmente. La ambición técnica debe escalar con la capacidad del equipo, no inversamente. Un proyecto con tres componentes bien hechos por un equipo de tres personas suele aportar más valor que un proyecto con diez componentes mal mantenidos por una sola.

8.2 Distribuir el conocimiento desde el inicio

Pair programming, *code reviews* sistemáticas, documentación viva y rotación de responsabilidades. El *bus factor* debe ser superior a uno desde el primer día. Si el conocimiento de un componente del sistema vive en una sola cabeza, ese componente es frágil estructuralmente, independientemente de su calidad técnica.

8.3 Definir criterios de cierre desde el inicio

Cada proyecto debería tener criterios explícitos de:

- **Éxito:** ¿cuándo se considera completado?
- **Pivote:** ¿cuándo cambiar de dirección o redefinir el alcance?
- **Cierre:** ¿cuándo concluir formalmente si las condiciones de viabilidad ya no se cumplen?

Estos criterios deberían estar documentados antes de empezar y revisarse periódicamente. Su ausencia es una de las causas principales de proyectos que se prolongan en estado moribundo.

8.4 MVP iterativo > arquitectura aspiracional

Empezar con la versión más simple que funcione *end-to-end* es preferible a diseñar la arquitectura completa por anticipado. Iterar añadiendo complejidad solo cuando se justifica con uso real, no con

anticipación. Esta es una de las lecciones más repetidas en la literatura de *lean startup* y se aplica igual a proyectos docentes que a empresariales.

8.5 Documentar para audiencias distintas

La documentación técnica para evaluadores no es la misma que la documentación interna para futuros mantenedores. La primera enfatiza arquitectura, decisiones y resultados; la segunda enfatiza cómo hacer cosas concretas, cómo levantar el proyecto, cómo debuggear. Crear ambas desde el inicio — y no como tarea final — multiplica la probabilidad de que el proyecto sobreviva la rotación de miembros.

8.6 Reconocer los límites de la asociación estudiantil

Algunas iniciativas requieren estructuras más permanentes que una asociación universitaria: grupos de investigación, *spin-offs*, o convenios formales de larga duración. Una asociación es excelente para aprender y prototipar; menos adecuada para operar producción a largo plazo. Identificar tempranamente cuándo un proyecto excede el formato asociativo permite buscar marcos institucionales más adecuados antes de generar deuda estructural.

Conclusión

El Proyecto Demeter ha cumplido sus objetivos didácticos: ha generado competencias técnicas en sus miembros, documentación reutilizable, un sistema funcional verificable y — como cierre — un conjunto de lecciones aprendidas que beneficiarán a futuros proyectos. Su cierre formal en este momento, lejos de constituir un fracaso, es la expresión madura del análisis de viabilidad realizado con la honestidad técnica que un Proyecto Docente requiere.

Apéndice A

Detalle Financiero

Este anexo recoge el desglose detallado de la inversión realizada en el Proyecto Demeter, complementando las tablas resumen del capítulo 4.

A.1 Inventario de hardware adquirido

El inventario se presenta en dos bloques según su trazabilidad documental. La **instalación hidráulica y de potencia** procede de un único expediente de compra con presupuesto formal y se detalla unidad a unidad. La **electrónica de control** se adquirió de forma incremental en distribución *online* durante el desarrollo, sin expediente unificado, por lo que se inventaría por tipo de componente y con precio de referencia de mercado.

Cuadro A.1: Instalación hidráulica y de potencia. Expediente único con trazabilidad completa.

Componente	Uds.	€/ud.	Total	Ubicación actual
Bomba eléctrica autocebada 12 V (Coolty)	4	29,99	119,96 €	Almacén ESIBot
Electroválvula DC 12 V 1 "	6	36,74	220,44 €	Almacén ESIBot
Válvula reguladora de presión 1 "	4	31,91	127,64 €	Almacén ESIBot
Válvula antirretorno 1 "	4	18,11	72,44 €	Almacén ESIBot
Barril 60 L boca ancha HDPE sin BPA	4	28,45	113,80 €	Almacén ESIBot
Kit de grifos para tanque	2	16,80	33,60 €	Almacén ESIBot
Fuente carril DIN 12 V 120 W	1	29,93	29,93 €	Almacén ESIBot
Cable plano gemelo 2 núcleos 10 m	1	25,99	25,99 €	Almacén ESIBot
Base imponible			743,80 €	
IVA (21 %)			156,20 €	
TOTAL EXPEDIENTE			900,00 €	

Trazabilidad del expediente: presupuesto nº **26317**, emitido el **31/10/2025** por *Juan Rodrigo Jiménez-Díaz* — *MR MICRO Sevilla* (NIF ES45659910B, C/ Feria 135, Local UPI, 41002 Sevilla) a

nombre de la **Escuela Técnica Superior de Ingeniería Agronómica** (Carretera de Utrera km 1, 41013 Sevilla; NIF Q4118001I), bajo la referencia **ESIBOT**. Forma de pago: transferencia bancaria. Material recibido y disponible desde **enero de 2026**.

Cuadro A.2: Electrónica de control y sensores.

Componente	Uds.	€/ud.	Total	Ubicación actual
ESP32-S3-DevKitC-1	3	~15	45 €	Almacén ESIBot
Raspberry Pi 4B (4 GB)	1	~75	75 €	Almacén ESIBot
MicroSD 32 GB clase 10	1	~9	9 €	Almacén ESIBot
Sensor DS18B20 (sonda impermeable)	10	~4	40 €	Almacén ESIBot
Sensor capacitivo de humedad de suelo	10	~3	30 €	Almacén ESIBot
Sensor DHT22	10	~5	50 €	Almacén ESIBot
Módulo relé 4 canales	3	~6	18 €	Almacén ESIBot
Convertidores TTL/USB	2	~5	10 €	Almacén ESIBot
Fuentes de alimentación 5 V	3	~5	15 €	Almacén ESIBot
Baterías LiPo 3000 mAh	2	~12	24 €	Almacén ESIBot
Carcasas/protecciones IP65	2	~10	20 €	Almacén ESIBot
Cables, jumpers, <i>breadboard</i>		lote	30 €	Almacén ESIBot
TOTAL ELECTRÓNICA			366 €	

Trazabilidad de la electrónica: adquirida en distribución *online* en pedidos pequeños a lo largo de 2025 y 2026, sin expediente de compra unificado ni número de factura único. Los importes son precios de referencia de mercado por unidad. El material está íntegramente depositado en el almacén de la asociación.

Identificación de equipos. El nodo *gateway* es identificable de forma unívoca por su dirección MAC 9C:13:9E:A8:6F:CC, que aparece codificada en el firmware de los nodos sensores como destino ESP-NOW. Los nodos restantes se identifican por su NODE_ID de compilación (1 *gateway*, 2 sensor, 3 actuador).

Inversión total en hardware: 900,00 € (hidráulica) + 366 € (electrónica) = **1.266 €**.

A.2 Histórico de gastos en infraestructura cloud

Cuadro A.3: Gastos mensuales en Google Cloud Platform.

Mes	Importe (€)
01/2026	25 €
02/2026	25 €
03/2026	25 €
04/2026	25 €
05/2026	25 €
06/2026	25 €
07/2026	25 €
08/2026	25 €
TOTAL ACUMULADO	200 €

El gasto se mantuvo constante en 25 €/mes porque la carga del sistema no variaba: una única máquina virtual encendida de forma permanente, con un volumen de telemetría muy por debajo de cualquier umbral de facturación por uso. La cifra de 200 € es **definitiva**: la cuenta ya ha sido cerrada y no habrá más cargos.

Conviene subrayar el dato más revelador de esta tabla: **el reloj de facturación no se detuvo cuando lo hizo el desarrollo**. El último *commit* del repositorio es de marzo de 2026, pero la infraestructura siguió facturando **cinco meses más** sin ningún trabajo asociado — 125 €, el 62 % del total gastado en cloud, se consumieron después de que el proyecto dejara de avanzar. Es la ilustración más directa del argumento del capítulo 4: el coste de un sistema desplegado es recurrente aunque el proyecto no lo sea, y solo deja de serlo cuando alguien lo apaga de forma deliberada.

Las facturas mensuales originales y los datos identificativos de la cuenta GCP del proyecto están disponibles bajo solicitud a través de la asociación.

A.3 Desglose detallado de horas-persona

Cuadro A.4: Estimación de horas dedicadas por miembro y área.

Miembro / Rol	Área	Horas estimadas
Daniel M. (lead)	Firmware ESP32	35
Daniel M. (lead)	Servicio RPi	15
Daniel M. (lead)	Backend FastAPI	40
Daniel M. (lead)	Frontend	20
Daniel M. (lead)	SDK + documentación	45
Otros miembros	—	0
TOTAL		155

Estas estimaciones son orientativas: la asociación no implementó sistema formal de *tracking* de horas, por lo que los valores se apoyan en el análisis de los sellos de tiempo del repositorio descrito en el capítulo 4.

La fila de *otros miembros* en cero no es una omisión, sino un dato: el repositorio registra **227 commits de un único autor** a lo largo de los 44 días naturales de desarrollo activo, sin ninguna

contribución de terceros al código, la documentación ni el despliegue. Es la constatación cuantitativa del *bus factor* igual a uno analizado en la sección 4.2.

A.4 Fuentes de financiación recibidas

Cuadro A.5: Fuentes de financiación recibidas durante el proyecto.

Fuente	Fecha	Importe
Universidad de Sevilla — Proyecto Docente ETSIA (expediente 26317, instalación hidráulica)	10/2025	900,00 €
Asociación ESIBot — material propio y aportaciones internas (electrónica de control)	2025–2026	366 €
Patrocinios y subvenciones externas	—	0 €
TOTAL FINANCIACIÓN RECIBIDA		1.266 €

Toda la financiación recibida tiene carácter de **inversión puntual en material**. Ninguna de las dos fuentes cubrió gasto corriente: los 200 € de infraestructura cloud acumulados hasta la fecha se han sufragado al margen de estas partidas, y no existe ninguna asignación comprometida para ejercicios futuros.

A.5 Plan de devolución / cesión de hardware

Cuadro A.6: Destino propuesto del hardware adquirido.

Origen	financia- ción	Material	Destino propuesto
Proyecto ETSIA (900 €)	Docente	Instalación hidráulica y de potencia (bombas, electroválvulas, válvulas, barriles, fuente DIN, cableado)	Puesta a disposición de la ETSIA como patrimonio universitario. Se propone su permanencia en el almacén de la asociación únicamente si la Escuela lo autoriza expresamente.
Asociación (366 €)	ESIBot	Electrónica de control, sensores y accesorios	Conservación en el almacén de la asociación y reaprovechamiento en futuros proyectos. Es material genérico, no específico de Demeter.
Aportaciones personales	—	—	No procede: no hubo aportaciones personales de material.

Situación actual: la totalidad del material, de ambos orígenes, se encuentra depositada en el **almacén de la asociación ESIBot**. La instalación hidráulica no llegó a montarse en campo, por lo que se conserva sin uso y en su estado original de recepción.

Acción propuesta: realizar el inventario físico contradictorio durante la semana 3 del cronograma del Anexo B, levantando acta firmada entre la asociación y la persona designada por la ETSIA para

la recepción, con la tabla [A.1](#) como base de comprobación unidad a unidad.

Apéndice B

Plan de Entrega de Artefactos

Este anexo detalla los artefactos a entregar como parte del cierre formal del Proyecto Demeter, organizados por naturaleza (digital, físico, conocimiento) y con cronograma asociado.

B.1 Artefactos digitales

Cuadro B.1: Artefactos digitales a entregar.

Artefacto		Detalle / Ubicación
Repositorio público		<code>github.com/DanielMf31/Proyecto_Demeter</code> (privado en el momento de redactar; se abre en la semana 1 del cronograma)
Documentación (PDF)	Técnica	<code>docs/Architecture/LaTeX/main.pdf</code> en el propio repositorio (84 páginas)
Documentación (fuentes LaTeX)	Técnica	Carpeta <code>docs/Architecture/LaTeX/</code>
Informe de Viabilidad (este documento)		Carpeta <code>docs/Closure/LaTeX/</code>
Backups de base de datos		No se entrega <i>dump</i> : la base solo contiene datos de prueba y semilla, reproducibles con <code>make seed</code> . El esquema viaja versionado en las migraciones de Alembic
Imágenes Docker		Ya publicadas en GitHub Container Registry (<code>ghcr.io/danielmf31/proyecto_demeter/{backend,frontend}</code>); pasan a públicas junto con el repositorio
Diagramas fuente (.mmd, .drawio)		Carpeta <code>docs/diagrams/</code> en repositorio

Licencias propuestas:

- Código: **MIT**. Se descarta Apache 2.0 porque su valor diferencial es la cláusula expresa de patentes, irrelevante en un proyecto sin cartera de patentes ni intención de litigio, mientras que MIT impone menos fricción a quien quiera reutilizar el código con fines docentes.
- Documentación: **CC-BY-SA 4.0** (atribución + compartir igual), de modo que las mejoras sobre el material docente reviertan en el mismo régimen abierto.

B.2 Artefactos físicos

Cuadro B.2: Plan de gestión del hardware físico.

Tipo de hardware	Ubicación actual	Destino propuesto
Instalación hidráulica (4 bombas, 6 electroválvulas, 8 válvulas, 4 barriles de 60 L, fuente DIN 12 V)	Almacén ESIBot	Puesta a disposición de la ETSIA: financiada con el Proyecto Docente, es patrimonio universitario
Nodos ESP32-S3 con sensores	Almacén ESIBot	Conservación en la asociación
Raspberry Pi 4B (gateway)	Almacén ESIBot	Conservación en la asociación
Cables, fuentes, accesorios	Almacén ESIBot	Reaprovechamiento en otros proyectos de la asociación
Carcasas y protecciones IP65	Almacén ESIBot	Reaprovechamiento
Baterías LiPo	Almacén ESIBot	Reaprovechamiento, previa comprobación de estado de carga

El detalle por origen de financiación se recoge en la tabla [A.6](#) del Anexo A.

B.3 Artefactos de conocimiento e infraestructura

Cuadro B.3: Gestión de cuentas externas e infraestructura.

Cuenta / Recurso	Estado actual	Acción propuesta
Cuenta Google Cloud Platform	Cerrada. La VM, el disco persistente y la IP reservada han sido eliminados; la facturación está detenida	Ninguna. Acción ya ejecutada: era el único gasto recurrente vivo del proyecto
Dominio <code>patata.monsters.org</code>	Subdominio del dominio personal del autor, no del proyecto	Retirar el registro DNS. No procede transferencia: el dominio raíz es ajeno al proyecto
Cuenta Cloudflare	Personal del autor; se usa solo como <i>tunnel</i> de acceso al entorno de <i>staging</i>	Eliminar el túnel de Demeter. La cuenta permanece con el autor
Documentación interna	Reside en el propio repositorio (<code>docs/</code> , 14.575 líneas de Markdown), no en una wiki externa	Se entrega con el repositorio; no hay plataforma de terceros que dar de baja
Repositorios privados	GitHub privado	Convertir a público al cierre
Credenciales y secretos	Vault privado del autor	Rotar / invalidar antes del cierre público

Consideración Importante

Antes de la apertura pública del repositorio es **obligatorio** revisar el histórico Git para asegurar que no contiene credenciales, claves API, contraseñas ni información personal sensible.

Resultado de la auditoría de secretos

La auditoría se ha realizado sobre los 227 *commits* del repositorio, revisando tanto el árbol actual como el histórico completo. Se resume a continuación.

Cuadro B.4: Hallazgos de la auditoría de secretos del repositorio.

Gravedad	Hallazgo	Acción requerida
Crítica	<i>Token</i> de túnel de Cloudflare válido en <code>.env.staging</code> , versionado desde su commit inicial y presente también en el árbol actual.	Revocar el túnel en Cloudflare antes de abrir el repositorio. Una vez revocado, el valor filtrado queda inservible y no es necesario reescribir el histórico.
Media	Ficheros <code>.env</code> de los cuatro entornos versionados, con contraseñas de PostgreSQL en claro.	Los valores son de desarrollo o marcadores explícitos (<code>change_me</code>), y la base de datos no es accesible desde el exterior. Basta con dejar de versionarlos y conservar únicamente <code>.env.example</code> .
Baja	<code>Core/auth.py</code> usa una clave de firma JWT por defecto codificada en el fuente si la variable de entorno no está definida.	No afecta al cierre, pero debe señalarse a quien reutilice el código: un despliegue que olvide definir la variable firmaría sus <i>tokens</i> con una clave pública y conocida.

No se han encontrado claves de cuenta de servicio de Google Cloud, claves privadas SSH o TLS, ni datos personales de terceros en ningún punto del histórico.

Con la revocación del *token* de Cloudflare — única acción bloqueante — el repositorio puede abrirse sin reescribir el histórico, lo que evita invalidar los identificadores de *commit* citados a lo largo de esta documentación.

B.4 Cronograma detallado

Cuadro B.5: Cronograma de entrega tras la aceptación del cierre.

Semana	Acciones
1	Auditoría de seguridad del repositorio (eliminación de secretos del histórico). Selección de licencias. Apertura del repositorio en GitHub público. Anuncio interno en la asociación.
2	Entrega formal de la documentación a la Universidad (PDFs + fuentes). Envío del informe de cierre completo. Reunión de cierre con el Departamento de Agronomía de la ETSIA, promotor del Proyecto Docente, y con la junta directiva de ESIBot.
3	Inventario físico definitivo del hardware. Devolución / cesión según origen de financiación. Firma del acta de entrega de hardware.
4	Baja del registro DNS y del túnel de Cloudflare. Acta de cierre técnico. El cierre de la infraestructura cloud (eliminación de la VM, el disco y la IP reservada) ya está ejecutado, por lo que esta semana solo recoge la formalización documental.

B.5 Documento de cierre formal

Como conclusión del proceso, se solicita la firma de un **acta de cierre** entre la Universidad de Sevilla y la Asociación Universitaria ESIBot, que recoja al menos:

- Reconocimiento de la conclusión del Proyecto Docente Demeter.
- Listado de artefactos entregados.
- Disposición del hardware adquirido con fondos institucionales.
- Reconocimiento del valor docente generado.
- Disponibilidad futura de los autores para consultas puntuales.
- Apertura a futuras colaboraciones bajo nuevos formatos.

Este acta constituye el cierre formal del proyecto y referencia institucional de la colaboración, facilitando que ambas partes mantengan relación operativa para futuras iniciativas.